

EE551 Mini-Project

Parking Spot Detector and Classifier

Parking Spot Detector and Classifier

- 55% of Overall Grade
- Due: 11:59 pm on Sunday 23rd November (end of second last week of Term)

Parking Spot Detector and Classifier

The “problem”: University of Galway wants to implement a system for parking permit holders. The system uses aerial drone footage to find empty car parking spaces. It updates the system as spaces are filled or become available.

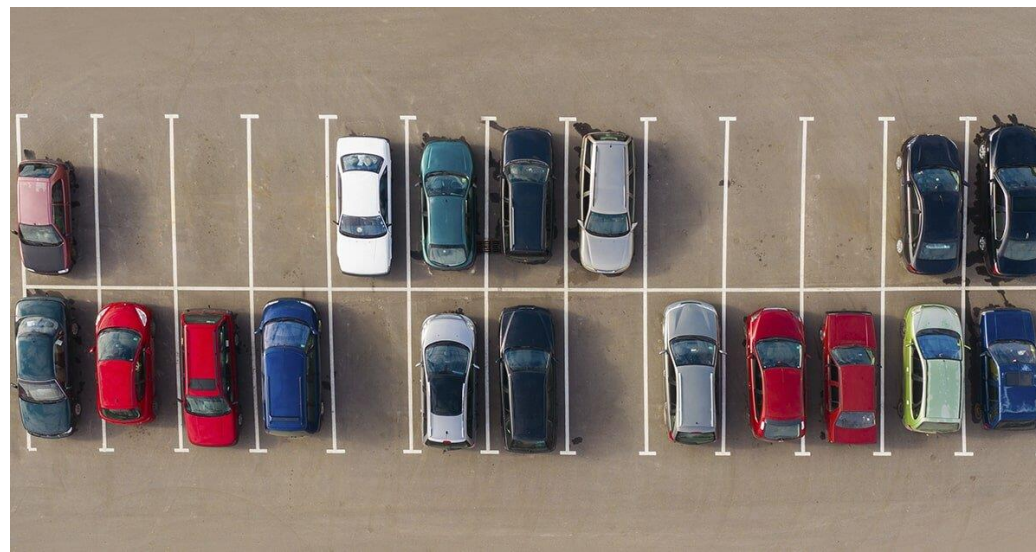
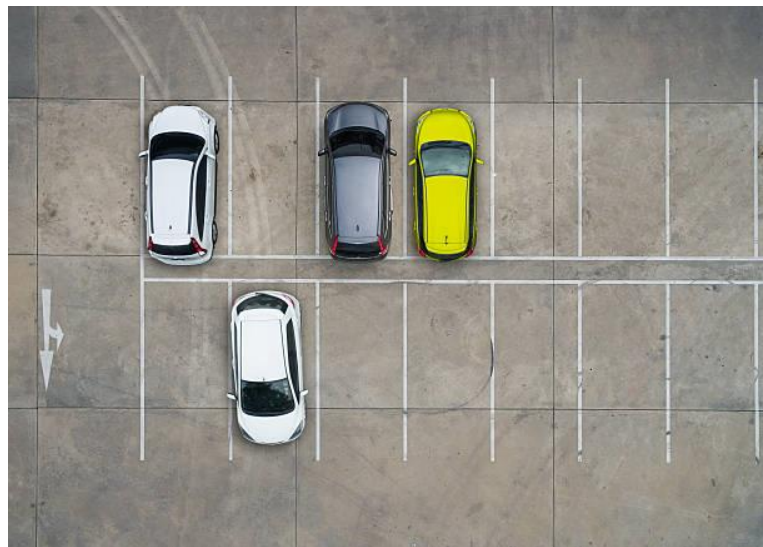
- Using Classical Image Processing methods, design a system which detects parking spaces and determines if they are occupied
- The system should accept a variety of image sizes and standard input types, .jpg, .png
- Consider implementation for real time application (computational cost etc.)



Classical Machine Vision (Why?)

- Less Computationally expensive, better for embedded applications. Can be run on low-power devices.
- Speed, can be optimised to produce outputs very quickly compared to Deep Learning methods

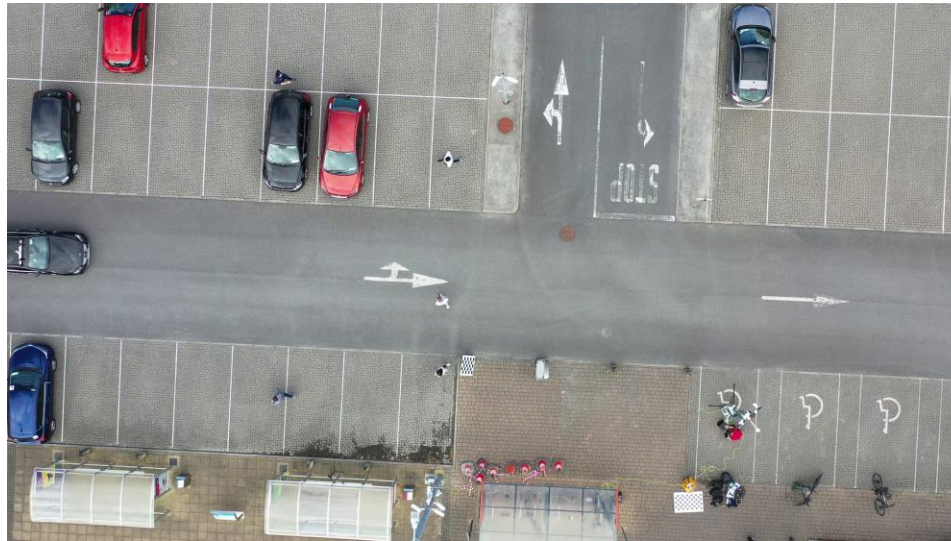
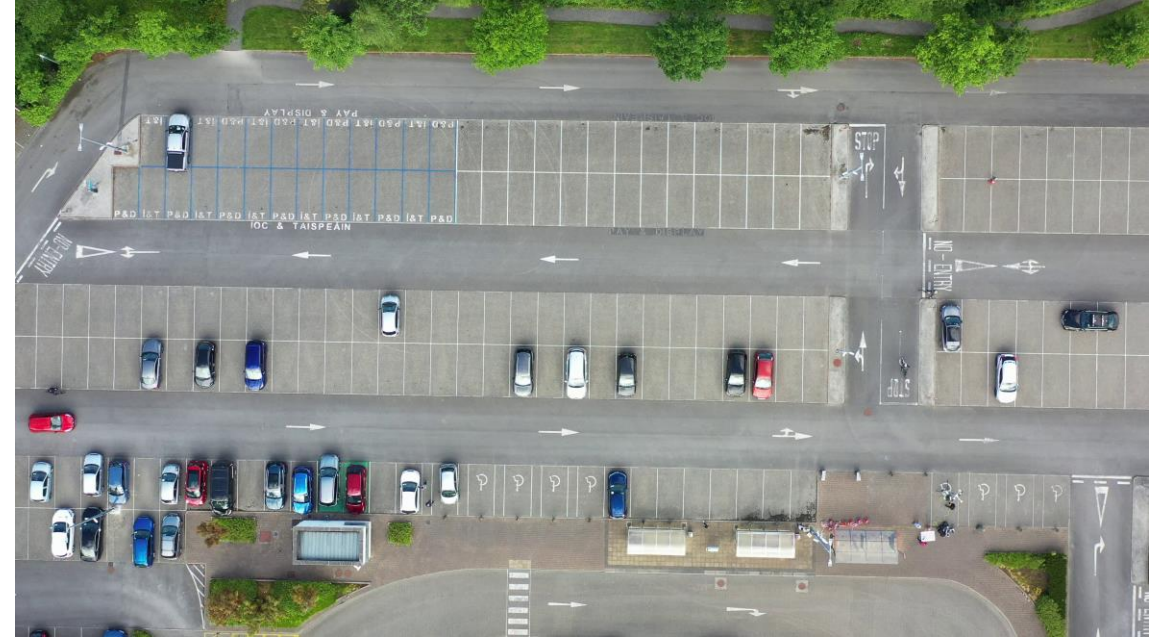
Input Images – “Easy”



Input Images – “Medium”



Input Images – “Hard”



*Not required to identify spots which are partially out of frame

Processing Pipeline (suggestions)

Consider using some of the following techniques:

- Greyscale
- Blur
- Edge Enhancement
- Contrast Enhancement
- Calculate Gradients
- Binarization
- Erosion + Dilation
- Canny Edge Detection
- SIFT/SURF feature detection
- RANSAC feature detection
- Harris Corner detection
- Hough Line Transform
- Filter Parking spaces

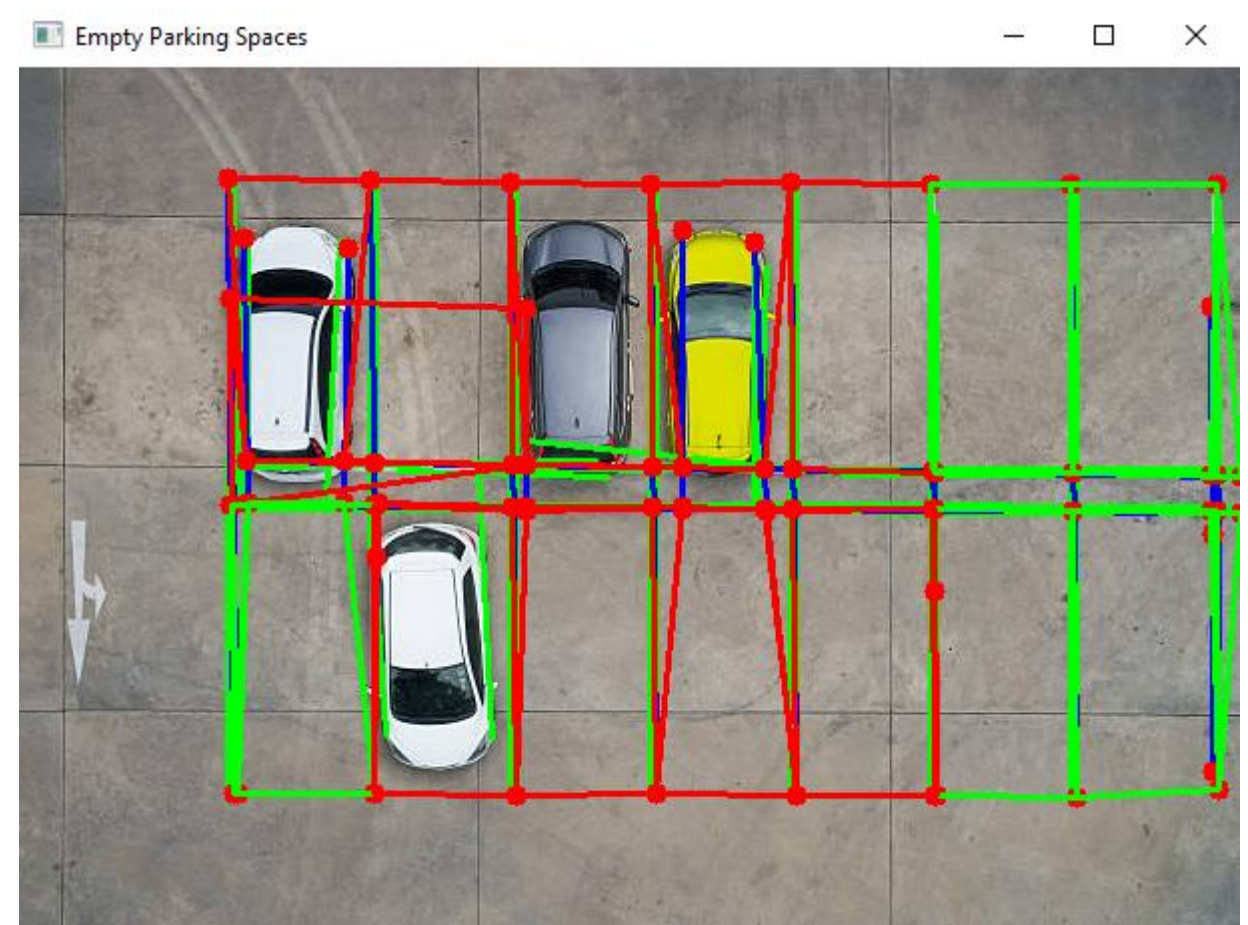
Tips:

- Use the image processing techniques from Assignments 1,2 & 3.
- Add Blocks incrementally, not all of them may be needed.
- Try using multiple iterations of a processing block.
- You may need to define your own functions to perform a niche processing step.
- It may not be the best approach to apply the blocks sequentially i.e. one after the other.
- Most likely a single solution will not work for every image, you will have to make changes to your code. Outline these changes in the report.

Output (base implementation Example)



Detected Parking Spaces (yellow)



Empty Spaces (green)

Submission Guidelines

- Submit Jupyter Notebook with your code, and a separate PDF report
- PDF (convert from word or latex): Use IEEE template [IEEE - Manuscript Templates for Conference Proceedings](#)
- Structure: Introduction, Methodology, Results, Discussion, Conclusion
- Include:
 - Final output for each image
 - Diagram of the processing pipeline.
 - Outline what each of the blocks does and how changing the parameters will influence the output.
 - Analysis of output – e.g. #detections/# actual spaces, false positives, missed detections
 - Analysis of embedded aspect, e.g. code optimisations, Big O analysis, performance different with down sampled images? Speed. Comparison against other methods such as a deep learning approach
- **No code in report.** Keep it high level: flow diagrams, images & equations.
- Critical analysis of solution – what worked well, what didn't work (and why), future work.
- In terms of how much technical detail to include: your target audience is you at the start of this module.
- Discussion section (outline challenges faced and how you dealt with them)
- Approx. 2-4 pages. Not a grading criteria !!
- Include citations where appropriate (websites, papers)